

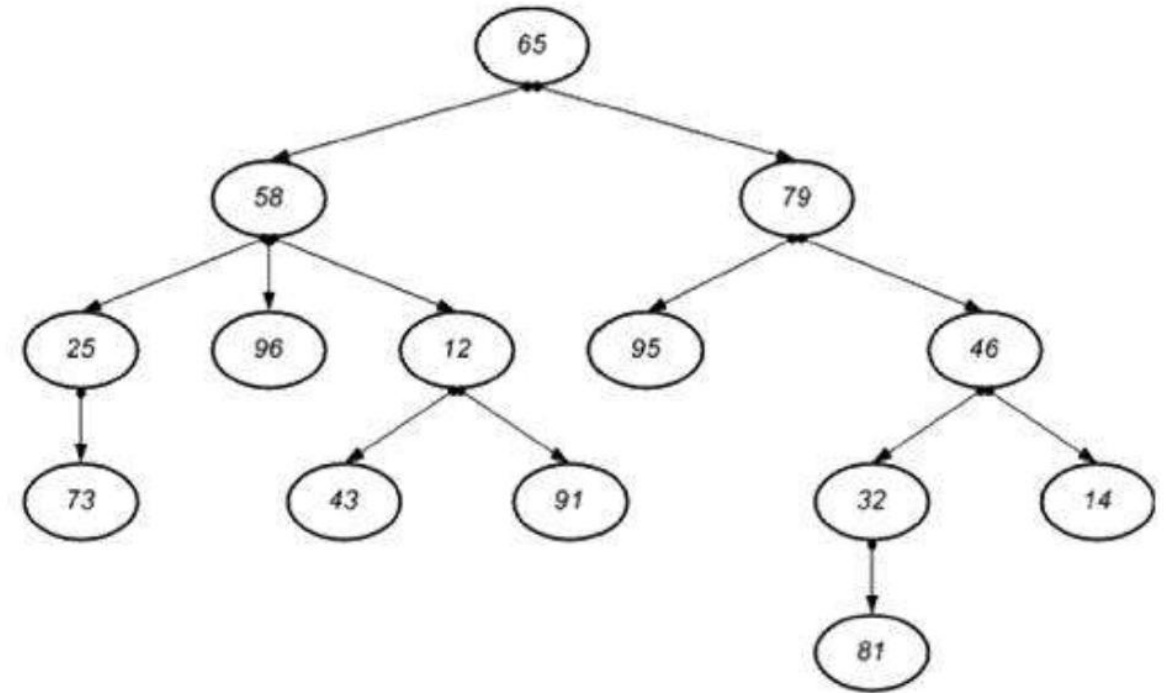


Árboles

Uso, características,
representación y construcción

Concepto

Un árbol es una estructura jerárquica aplicada a una colección de elementos u objetos llamados nodos. Uno de los cuales es conocido como raíz, creando una relación con los demás elementos lo cual da lugar a términos como padre, hijo, hermano, antecesor, sucesor, ancestro, etc.



Esquemáticamente la siguiente figura es la representación más usual y común de un árbol.

Características y propiedades de los árboles

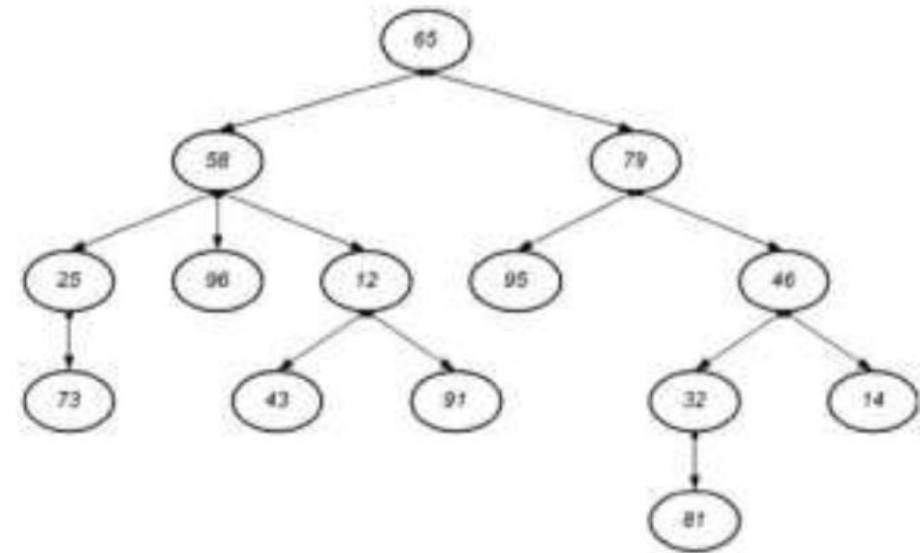
Término	Descripción
Nodos	Elementos o vértices de un árbol
Raíz	Todo árbol que no es vacío tiene un único nodo raíz, del cual descienden los demás elementos de un árbol.
Padre	Antecesor o ascendiente de un nodo, excepto nodo raíz.
Hijos	Descendientes de un nodo, puede ser varios.
Grado	Numero de hijos que salen de un nodo, es decir el numero de descendientes directos.
Nodo terminal u hoja	Todo nodo que no tiene ramificaciones (hijos) o con grado 0.

Características y propiedades de los árboles

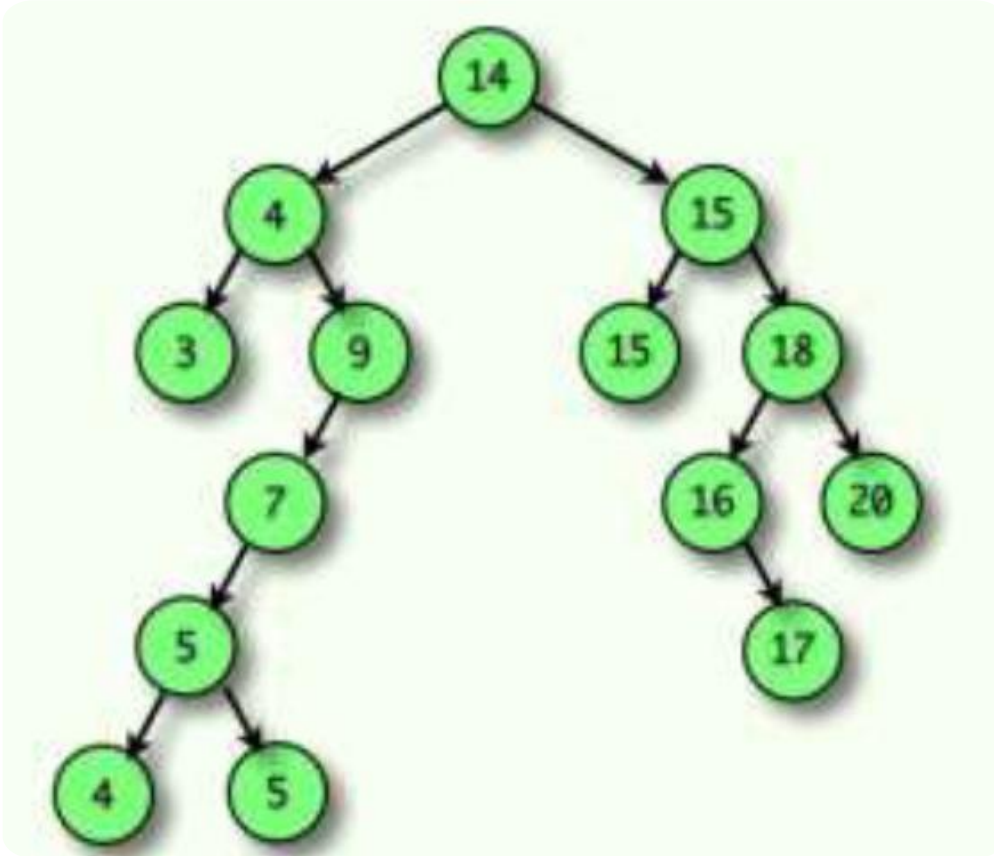
Término	Descripción
Hermanos	Todos los nodos que son descendientes directos de un mismo nodo.
Nivel	Número de antecesores que tienen un nodo desde la raíz, considerando que el nivel de la raíz es 1.
Profundidad o altura	Es el máximo de los niveles de los nodos de un árbol.
Peso de un árbol	Es el numero de nodos terminales.
Nodo interior	Todo nodo que no es raíz, ni hoja terminal
Grado de un árbol	Es el máximo grado de todos los nodos del árbol.

Termino	Algunos resultados
Nodos	
Raiz	
Padre	
Hijos	
Grado	
Nodo terminal u hoja	
Hermanos	
Nivel	
Profundidad o altura	
Pero del árbol	
Nodo interior	
Grado del árbol	

Ejemplo.
Considerando el
siguiente árbol.



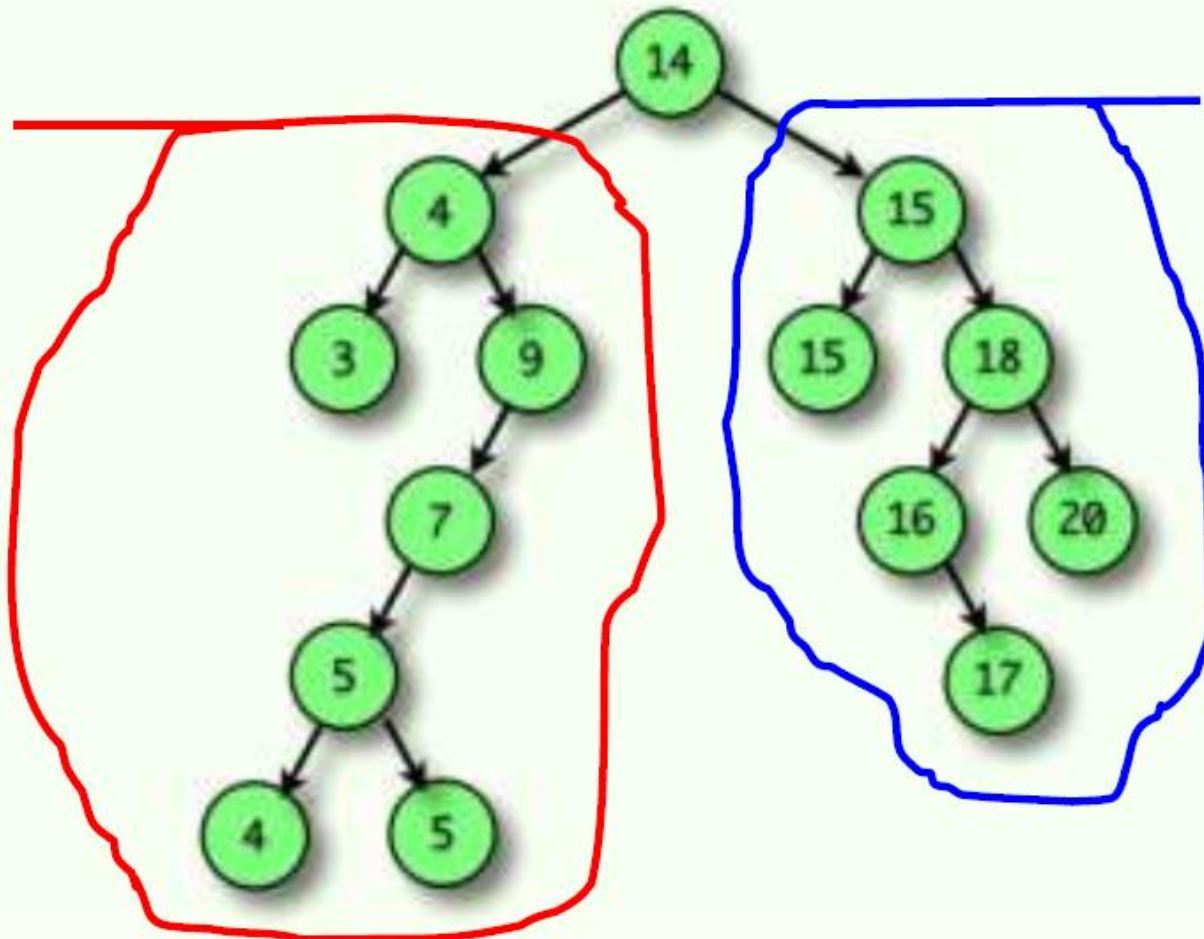
Arboles binarios

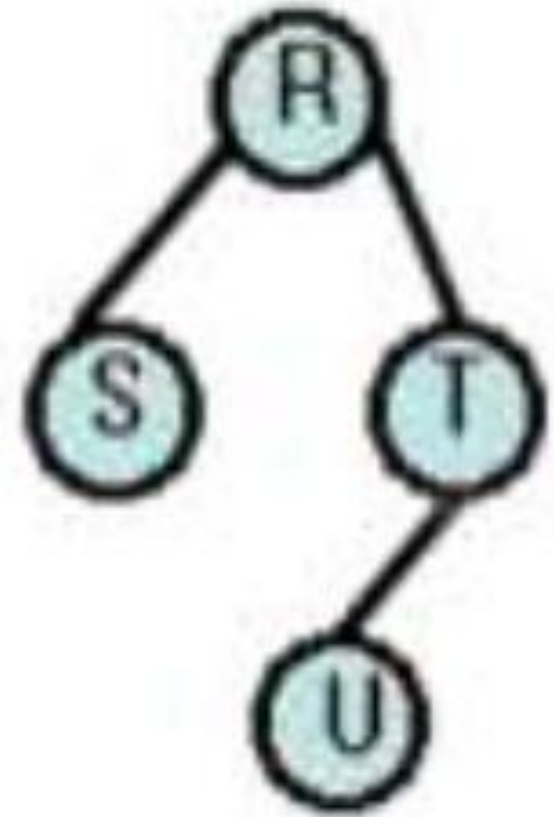
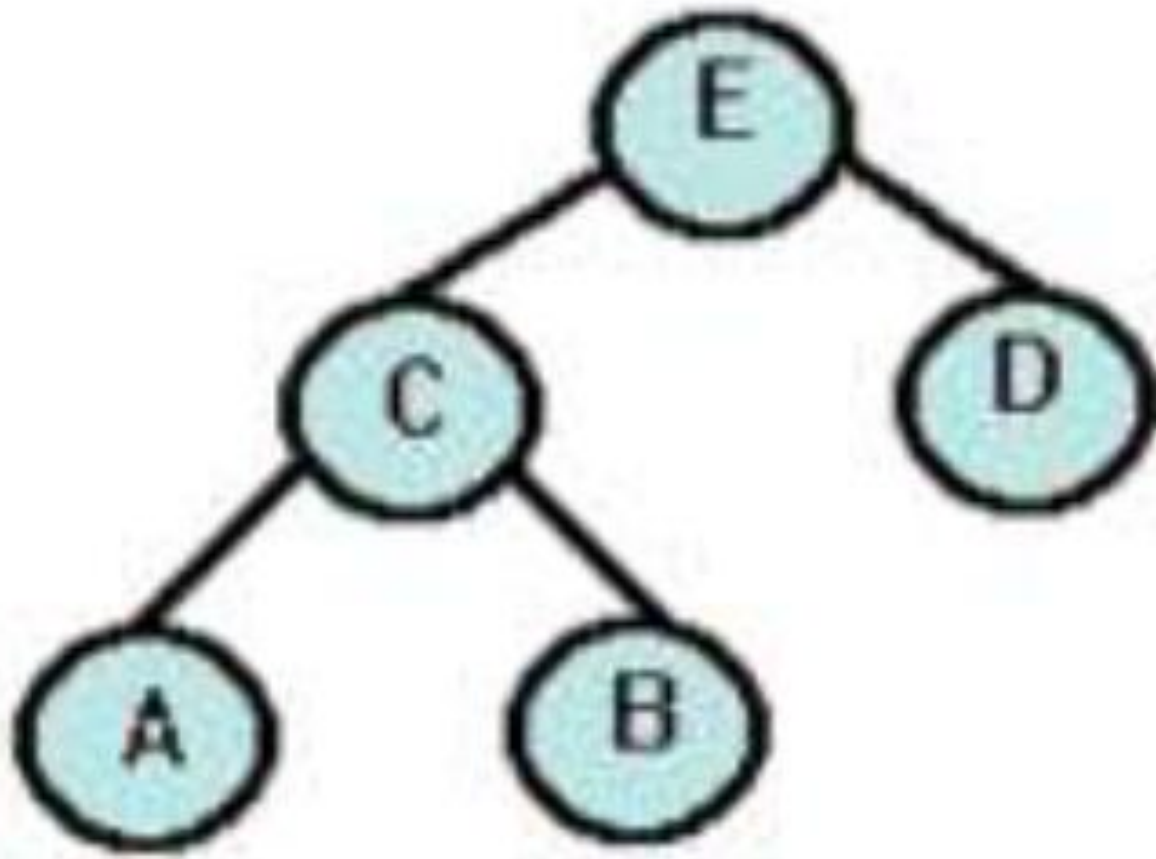


- Un *árbol binario* es aquél en el que cada uno de sus nodos no puede tener más de dos nodos hijos, es decir, dos subárboles identificados como *izquierdo* y *derecho* respectivamente, es un árbol que tiene como máximo el grado 2.
- Un *árbol binario* es un conjunto de nodos que constan de un nodo **raíz** enlazado a dos árboles binarios disjuntos (que ningún nodo puede estar en ambos subárboles) denominados **subárbol izquierdo** y **subárbol derecho**.

Subárbol izquierdo

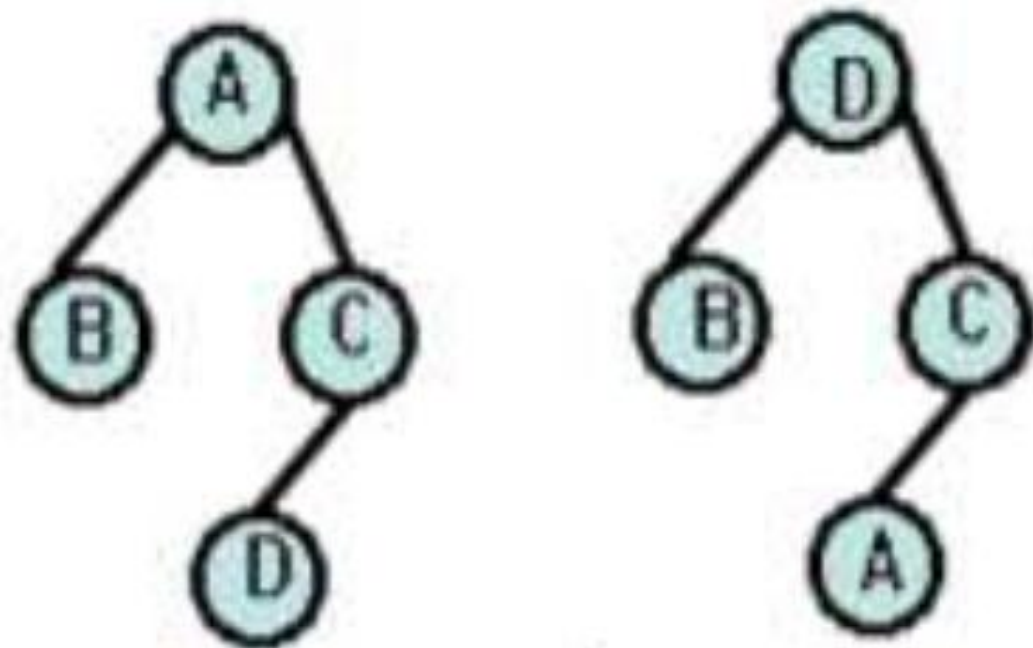
Subárbol derecho





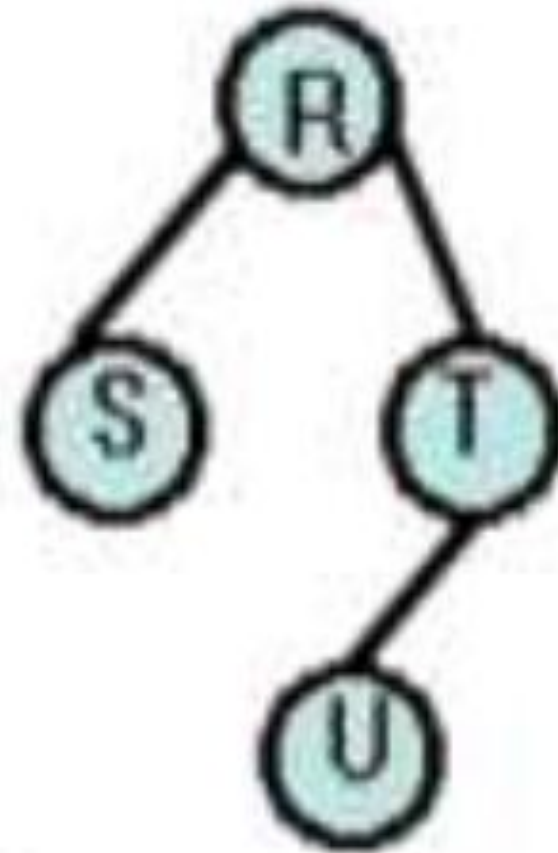
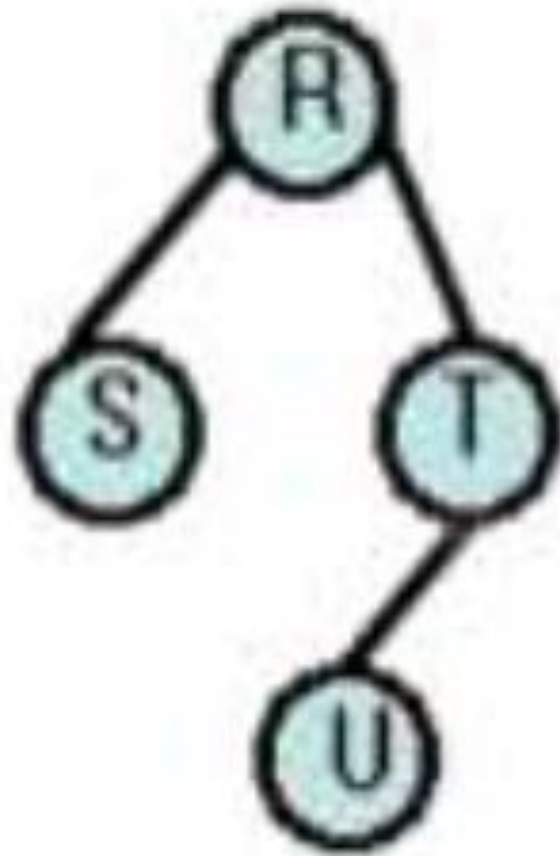
ÁRBOLES BINARIOS DISTINTOS, SIMILARES Y EQUIVALENTES

Distintos: cuando sus
estructuras son diferentes.



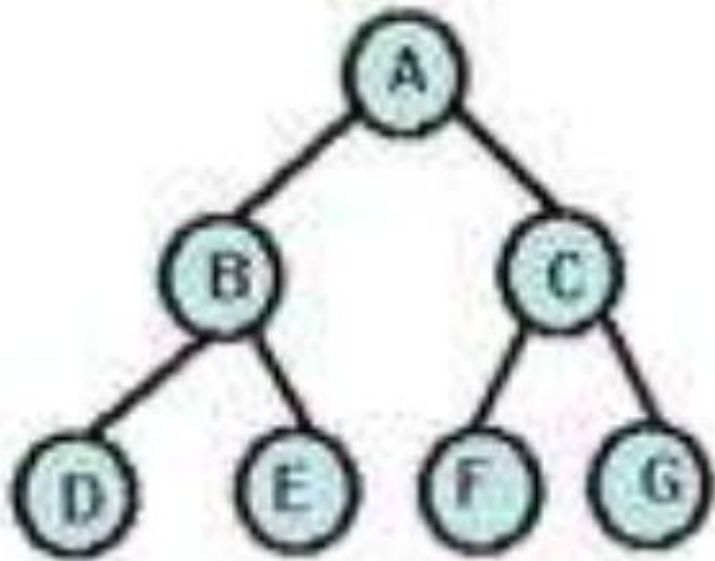
Similares: cuando sus estructuras son idénticas pero la información que contienen sus nodos difieren entre sí.

ÁRBOLES BINARIOS DISTINTOS, SIMILARES Y EQUIVALENTES

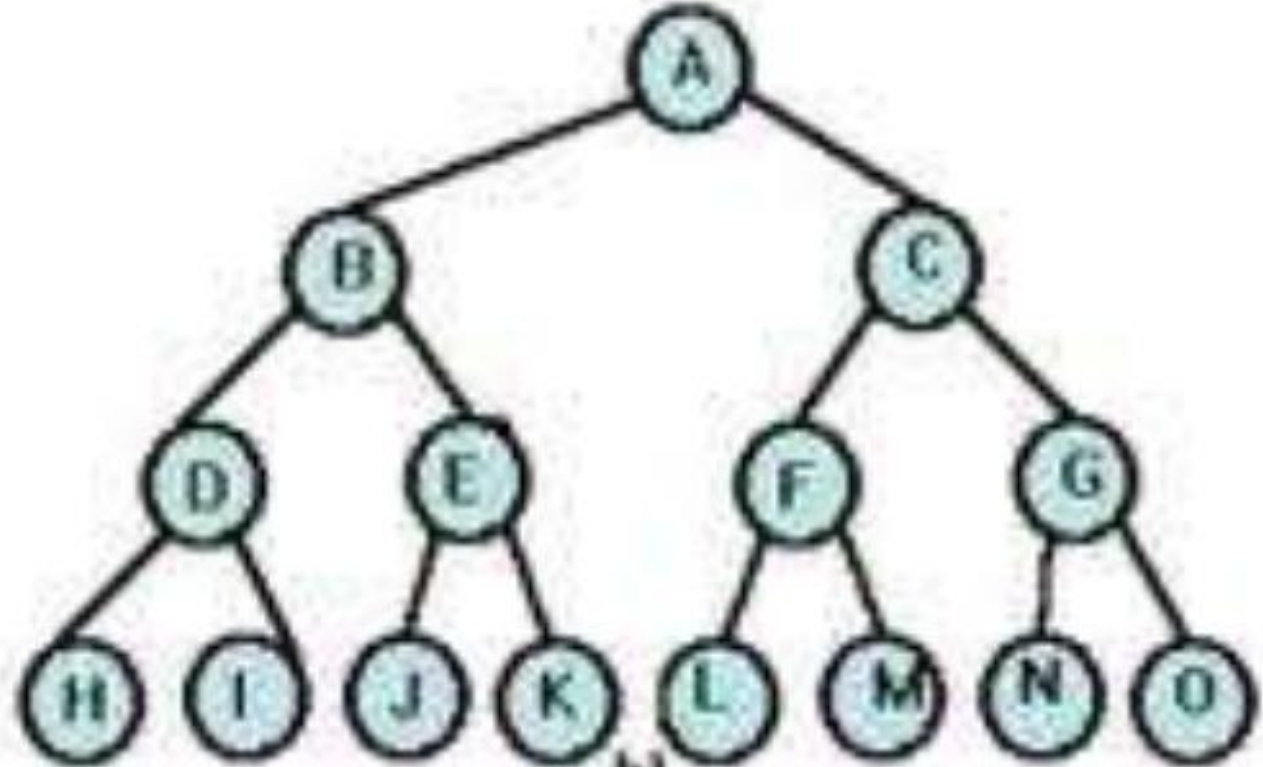


ÁRBOLES BINARIOS DISTINTOS, SIMILARES Y EQUIVALENTES

Equivalentes: aquellos que son similares y además los nodos contienen la misma información.



a)



b)

ÁRBOL BINARIO COMPLETO

Actividad 41

Es un árbol en el que todos sus nodos excepto los del último nivel tienen dos hijos.

```
class Node:
    def __init__(self, data):
        self.data = data          # Assigns the given data to the node

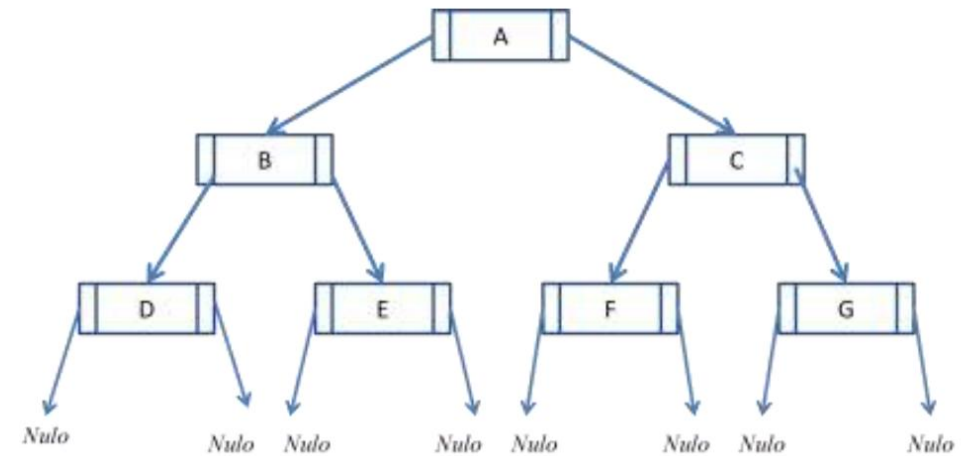
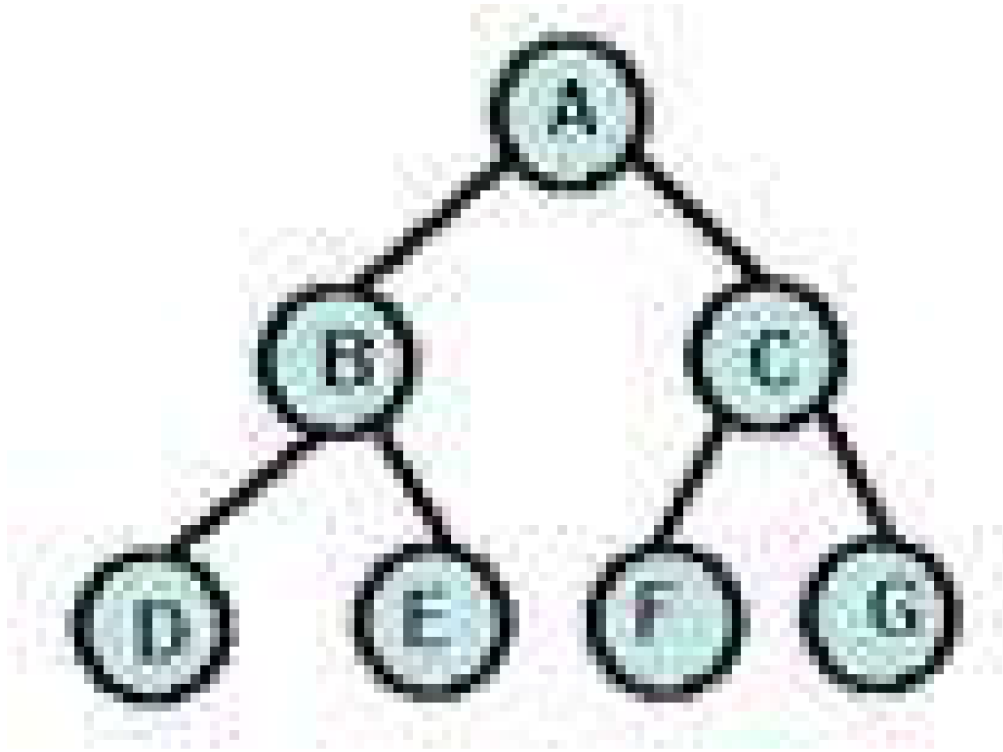
        self.izq = None          # Initialize the izq attribute to null
        self.der = None          # Initialize the der attribute to null
```

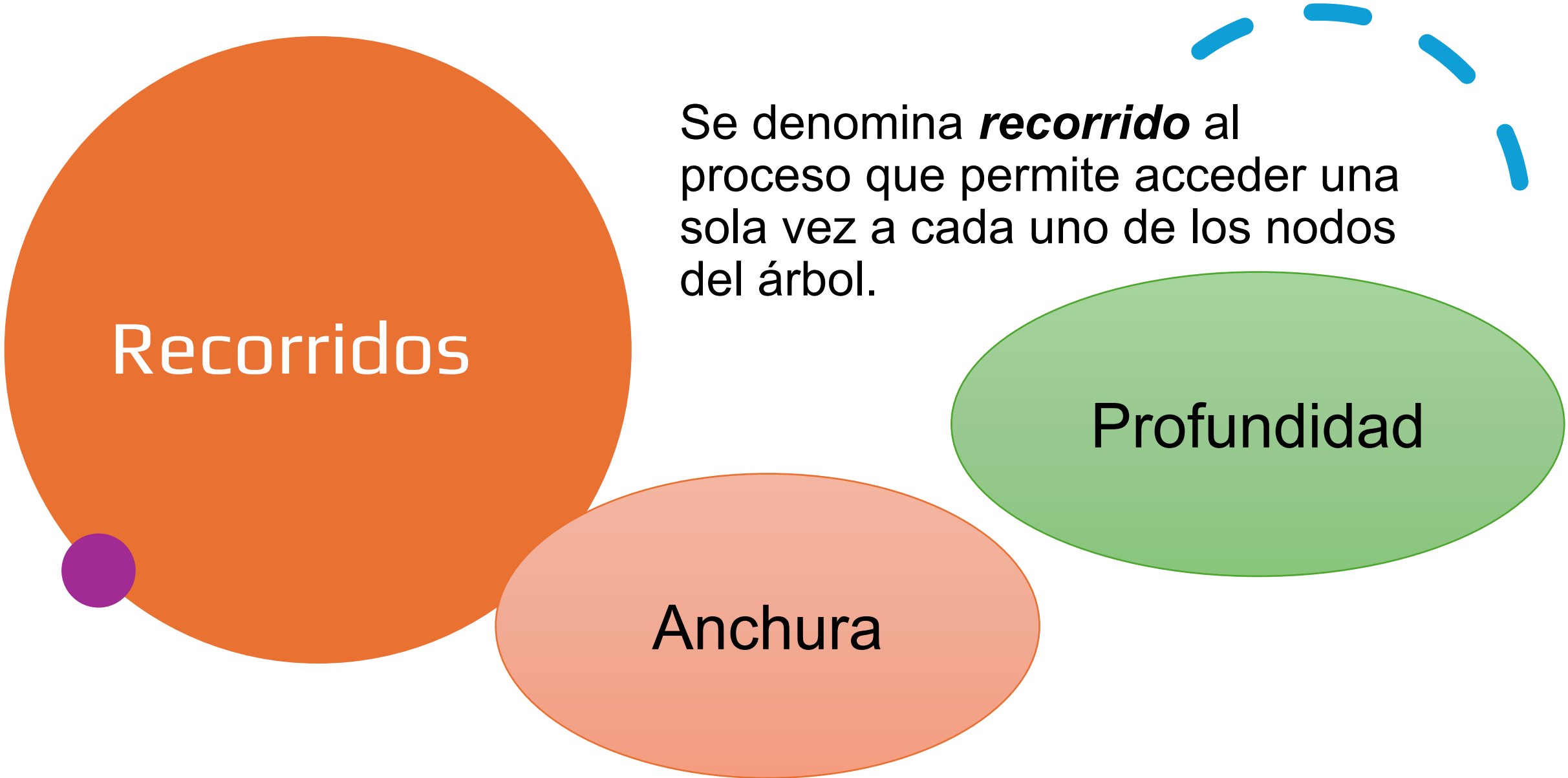
Izq	Info	Der
------------	-------------	------------

Implementación de un árbol binario

- Los nodos de un árbol binario están principalmente representados por registros, los cuales contienen como mínimo tres campos: *Izq*, *Info* y *Der*.

Por ejemplo, la representación de la siguiente figura con la definición del nodo anterior sería:





Recorridos

Se denomina ***recorrido*** al proceso que permite acceder una sola vez a cada uno de los nodos del árbol.

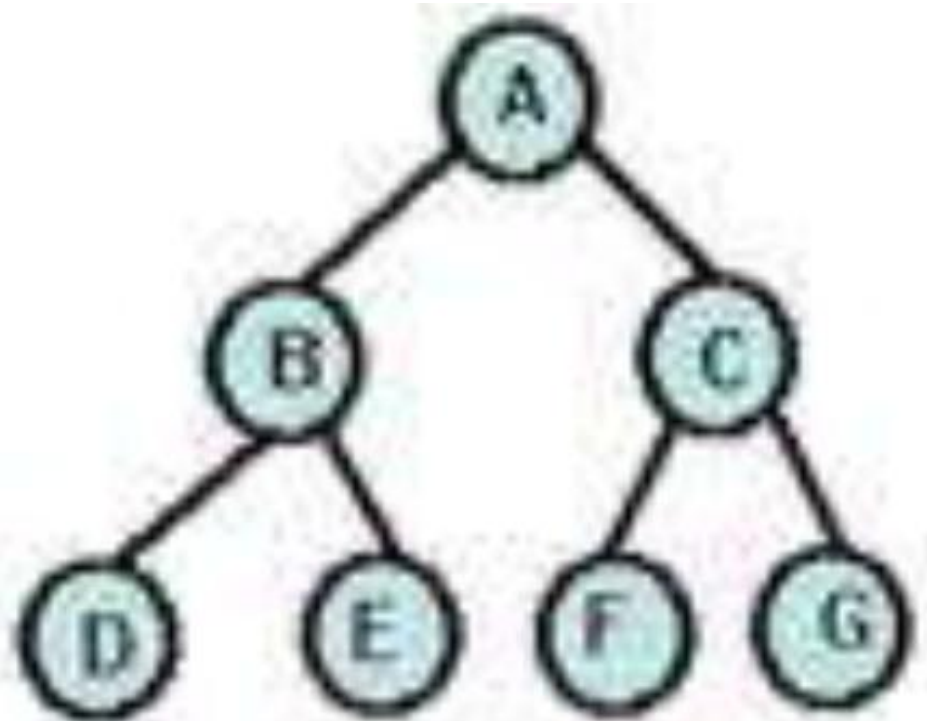
Anchura

Profundidad

Recorrido en Anchura

Consiste en recorrer los distintos niveles y, dentro de cada nivel, los diferentes nodos de izquierda a derecha.

A B C D E F
G



Recorrido en Profundidad

El árbol puede ser recorrido en diferentes órdenes, de acuerdo con el momento en que visita su *raíz*. De esta manera, se clasifican en tres tipos de recorridos:

PreOrden:
Raíz- Izquierdo-Derecho
(RID)

PostOrden:
Izquierdo-Derecho-**Raíz**
(IDR)

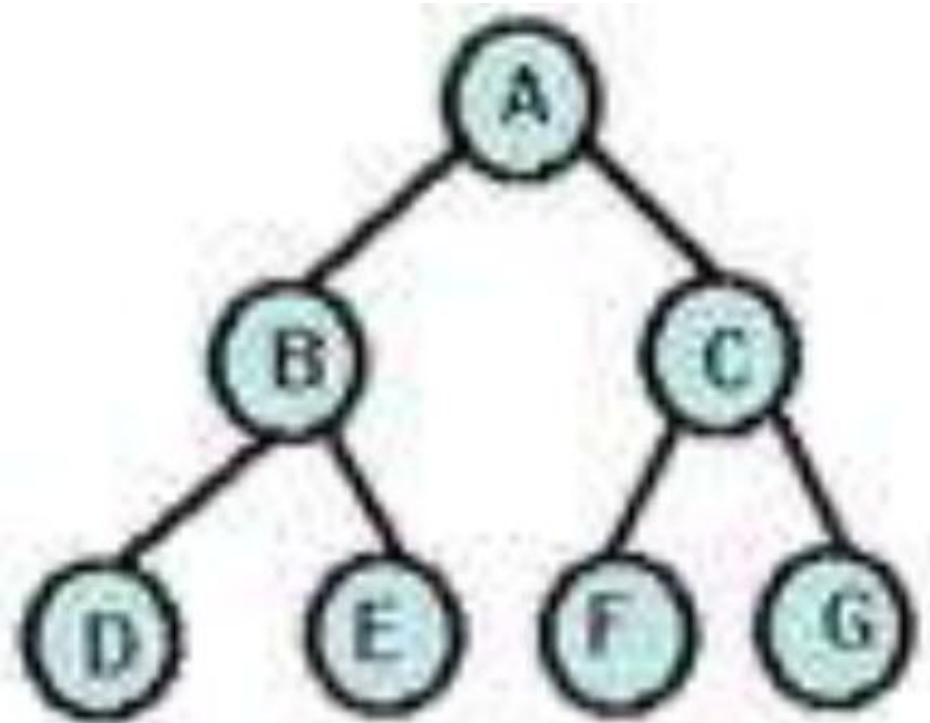
InOrden:
Izquierdo-**Raíz**-Derecho
(IRD)

Por ejemplo, de la figura anterior, los nodos visitados de acuerdo con su recorrido son:

PreOrden (RID): A B D E C F G

InOrden (IRD): D B E A F C G

PostOrden (IDR): D E B F G C A



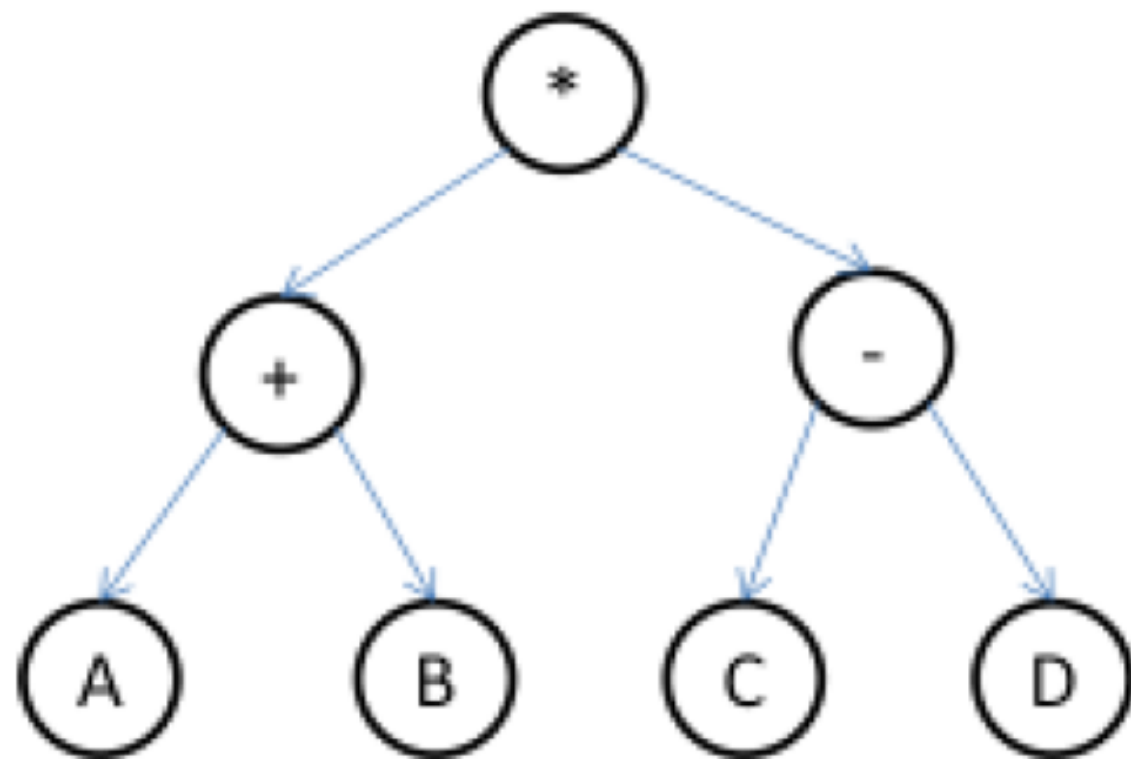
Árbol binario de expresiones

Una de las principales aplicaciones de un árbol binario es la de almacenar expresiones aritméticas en memoria. Ésta se puede representar considerando que toda expresión tiene operadores y operandos, considerando esto, el **operador** se representa en la *raíz* y los **operandos** en los nodos *izquierdo* y *derecho* según sea la expresión, lo cual quiere decir que los árboles de expresión son árboles binarios, cuyas hojas contienen operandos y los otros nodos operadores.

Por ejemplo, la siguiente expresión

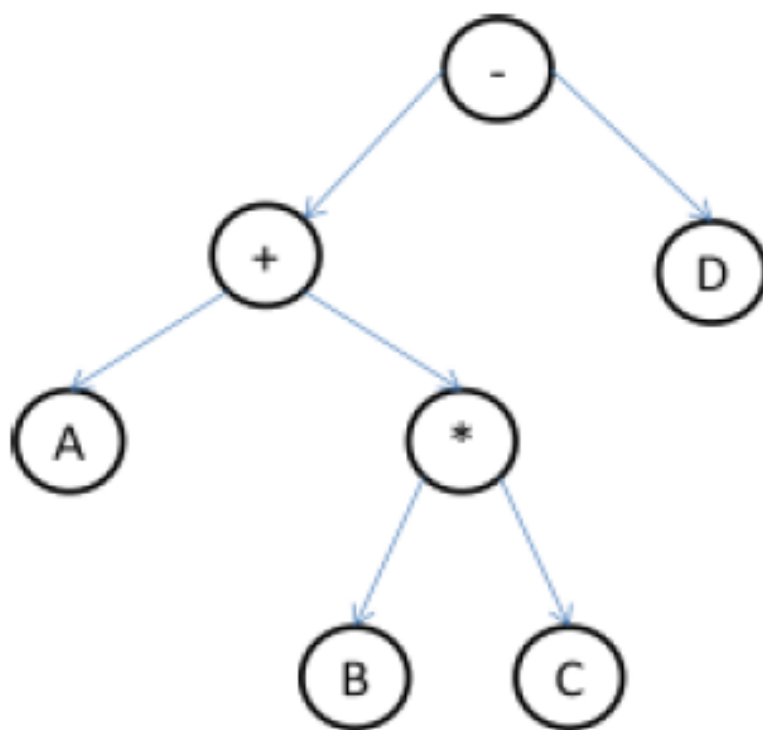
$$(A + B) * (C - D)$$

Tiene su presentación en árbol:



Esto se realiza utilizando la precedencia de los operadores y la asociación de los paréntesis, lo que significa que la siguiente expresión (sin paréntesis) no es lo mismo que la anterior.

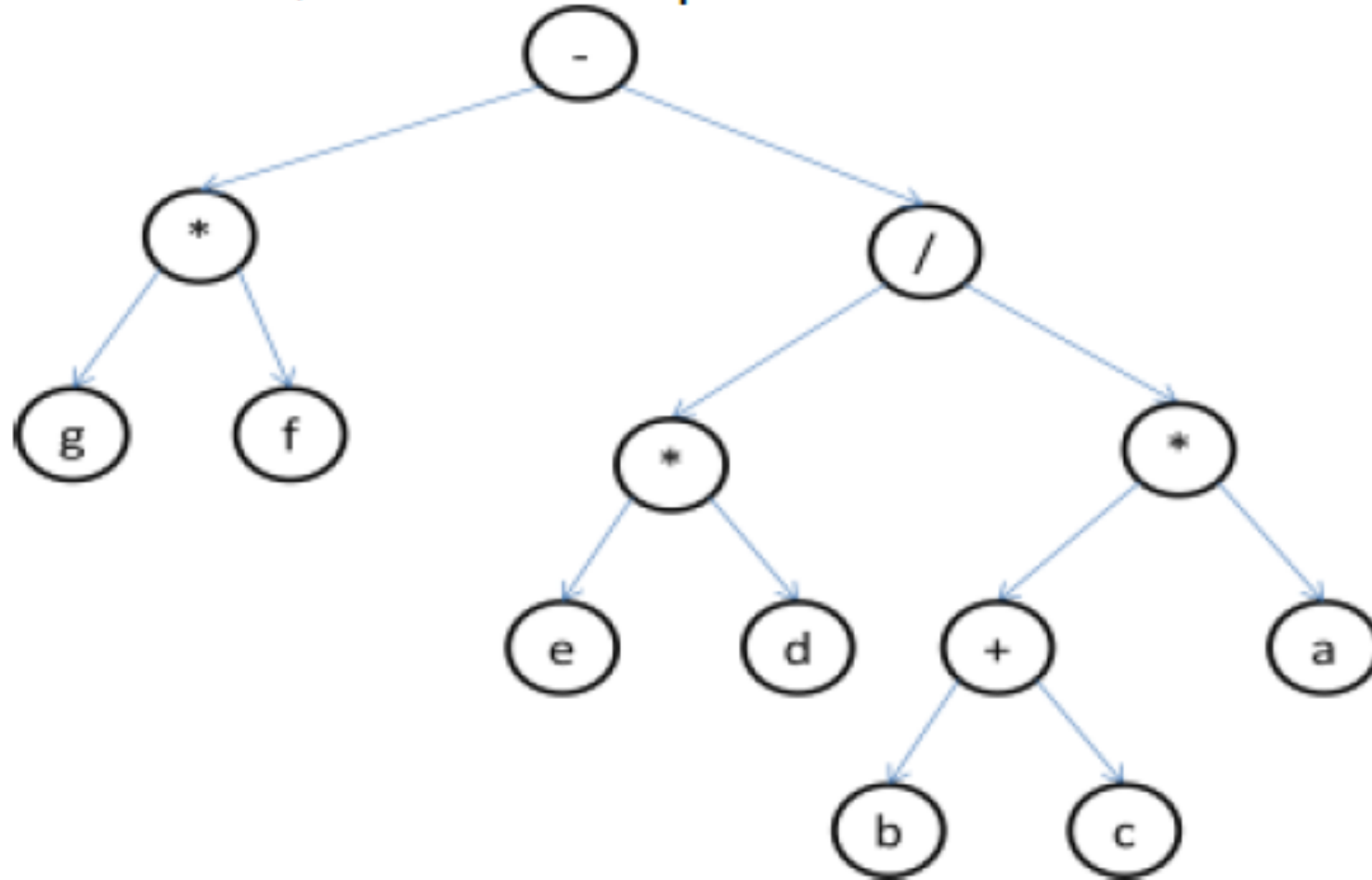
$$A + B * C - D$$



Así como a partir de una expresión se obtiene el árbol, se puede también de un árbol obtener su expresión.

Ejercicio

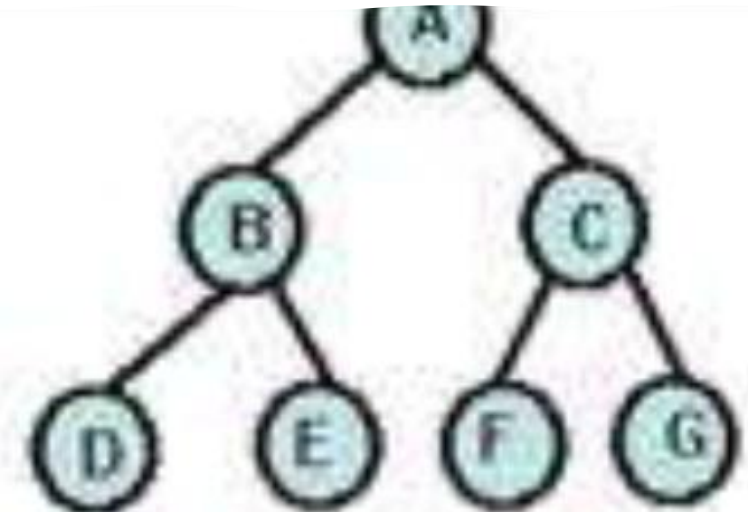
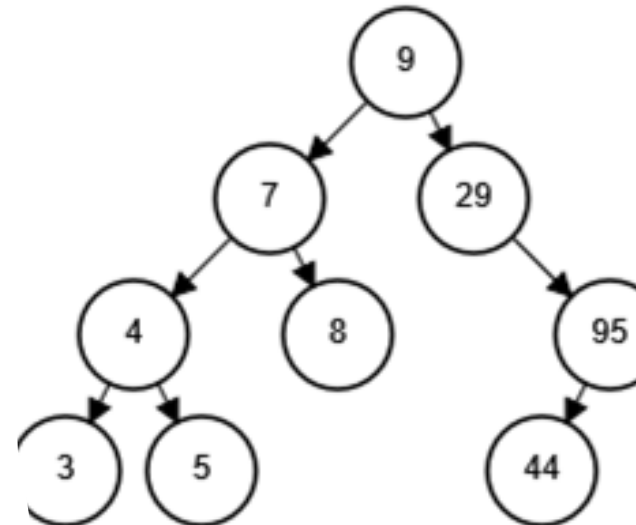
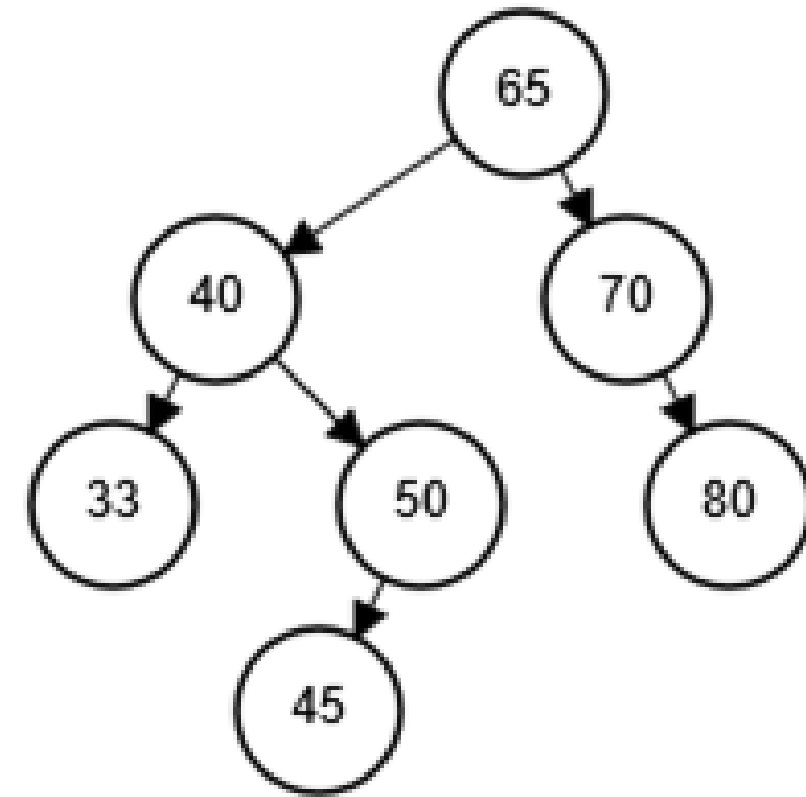
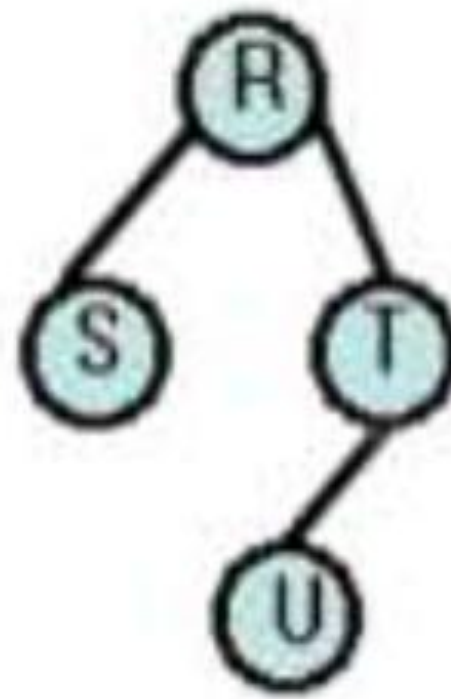
Se tiene el siguiente árbol, obtener su expresión.



$$(g*f) - [(e*d) / [(b+c)*a]]$$

ÁRBOLES BINARIOS DE BÚSQUEDA (OPERACIONES – INSERCIÓN, ELIMINACIÓN, BÚSQUEDA).

Un árbol binario de búsqueda es aquel en el que para cualquier nodo su valor es *superior* a los valores de los nodos de su subárbol izquierdo e *inferior* a los de su subárbol derecho, esto significa que este tipo de árbol es un árbol bien ordenado.

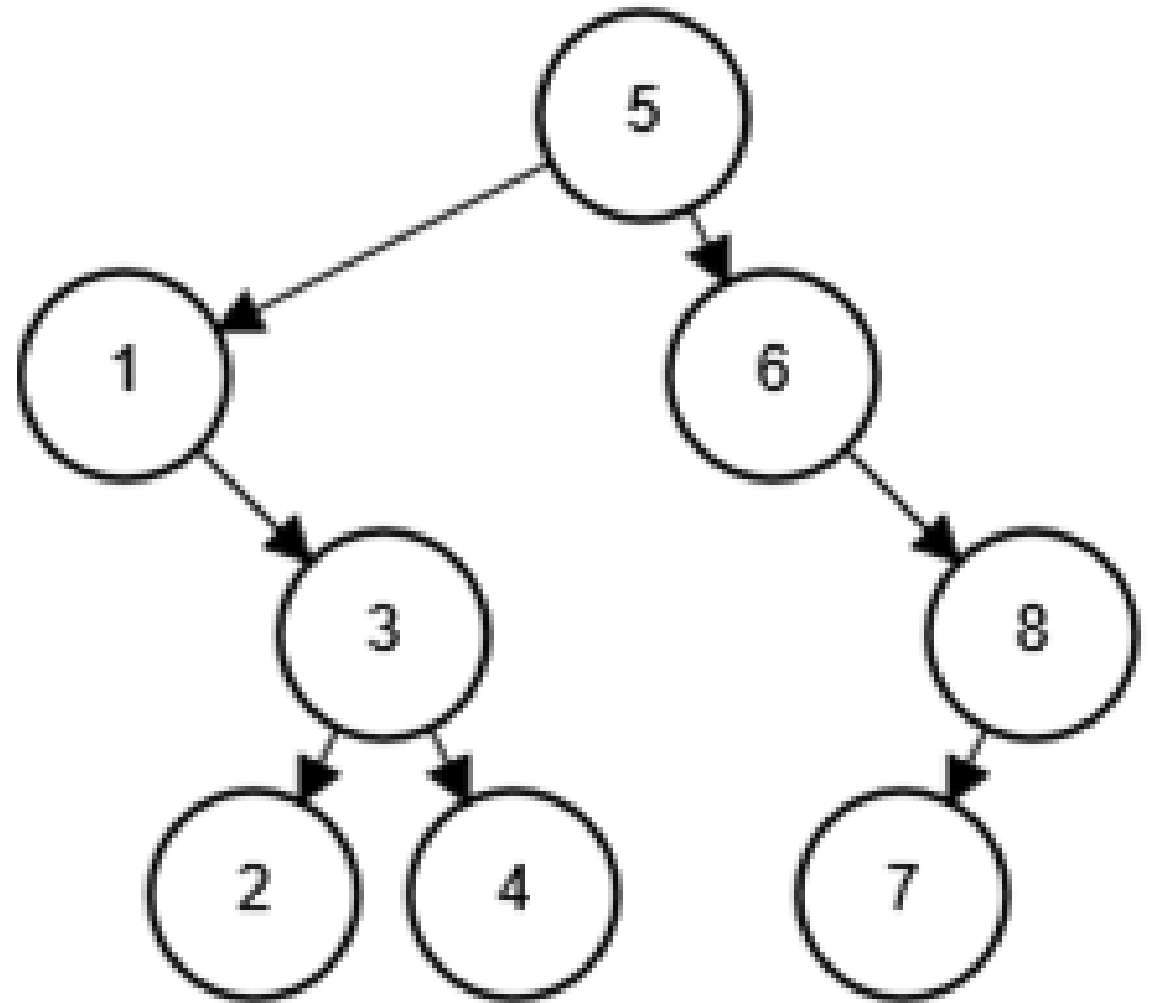


OPERACIONES: INSERCIÓN, ELIMINACIÓN Y BÚSQUEDA

La utilidad de este tipo de árbol se refiere a la eficiencia en la búsqueda de un nodo, similar al método de búsqueda binaria, y a la ventaja que presentan los árboles sobre los arreglos en cuanto a que la inserción y borrado de un elemento no requiere el desplazamiento de todos los demás

Búsqueda de un nodo

- La búsqueda de un elemento comienza en el nodo *raíz* y sigue los siguientes pasos:
 - El dato buscado se compara con el dato del nodo raíz.
 - Si los datos son iguales, la búsqueda se detiene.
 - Si el dato buscado es mayor que el dato de la raíz, la búsqueda se reanuda en el subárbol derecho. Si el dato buscado es menor, la búsqueda se reanuda en el subárbol izquierdo.



Buscar
4,8,2

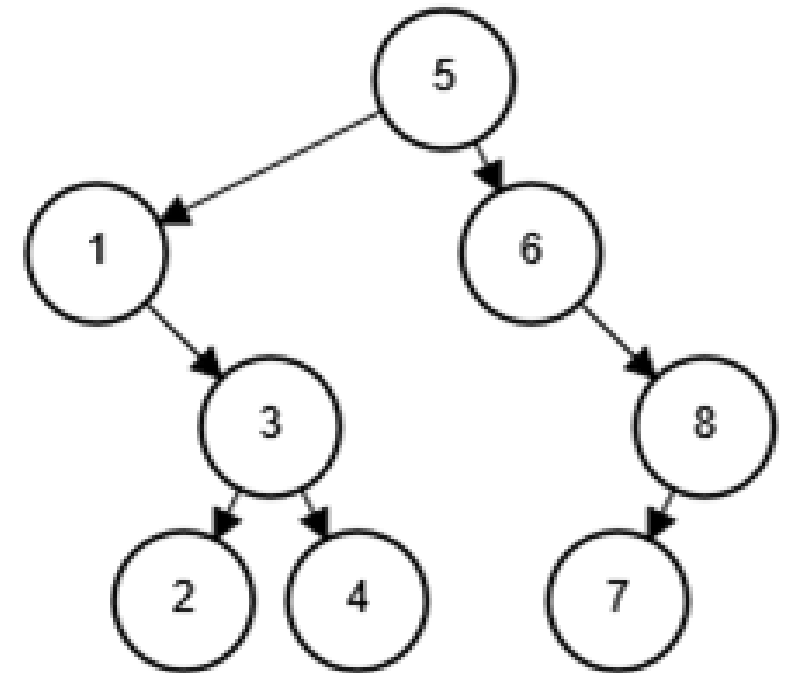
Inserción de un nodo

- El algoritmo que se sigue para la inserción de un nodo es:
- Comparar el dato del elemento a insertar con el dato del nodo raíz, si es mayor avanzar hacia el subárbol derecho, si es menor hacia el izquierdo.
- Repetir el paso anterior hasta encontrar un elemento con el dato igual o llegar al final del subárbol donde debiera situarse el nuevo elemento.
- Cuando se llega al final es porque no se ha encontrado, por tanto, se deberá reservar memoria para una nueva estructura de nodo, introducir los datos y asignar nulo a los punteros izquierdo y derecho de este. A continuación se colocará el nuevo nodo como hijo izquierdo o derecho el anterior según sea el valor del dato.

Ejemplo.

Del siguiente conjunto de números obtener su árbol binario de búsqueda

5 1 3 6 4 8 2 7

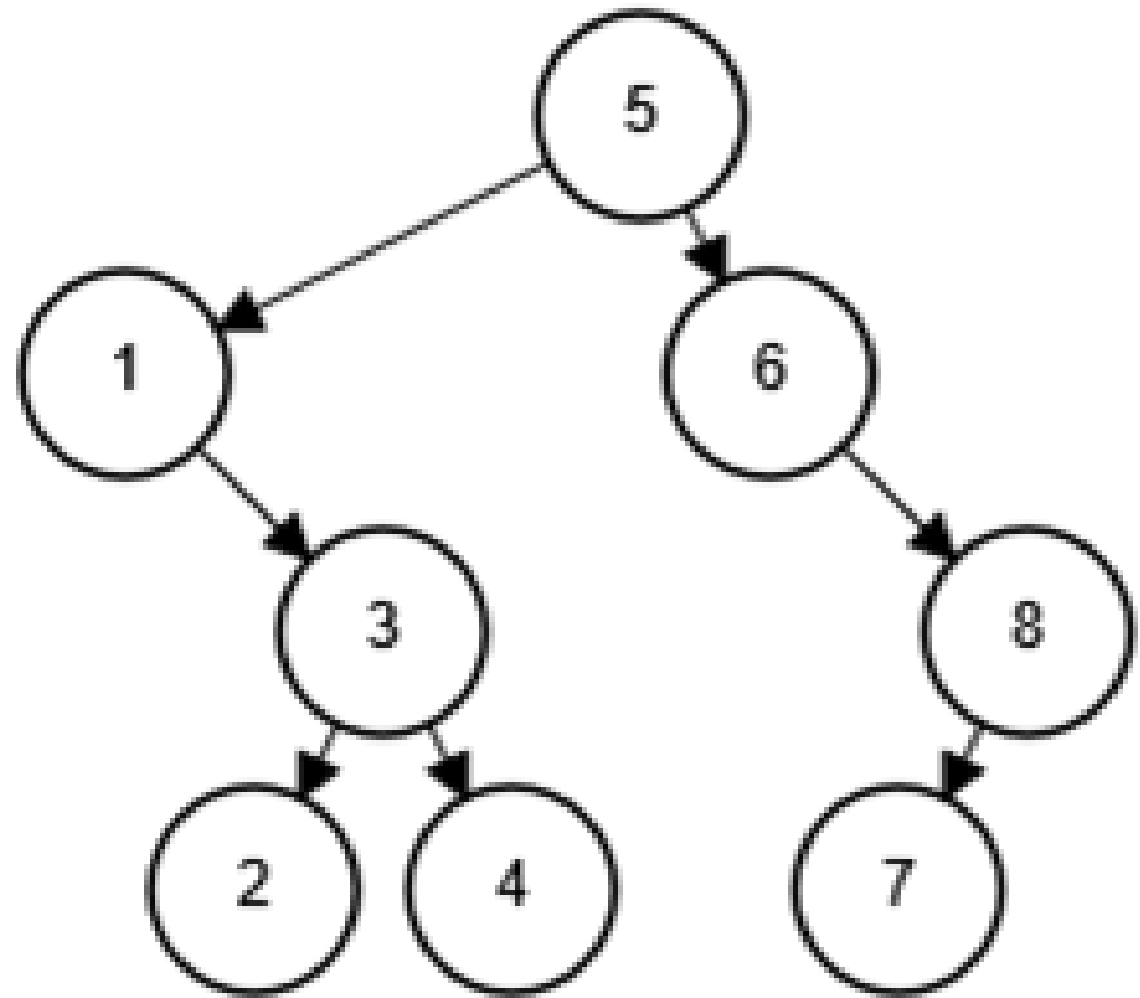


Eliminación de un nodo

- La eliminación o borrado de un nodo, después de haberlo encontrado requiere considerar las siguientes posibilidades:
 - Que no tenga hijos, es decir que sea hoja.
 - Se suprime, asignando nulo al puntero de su antecesor que lo apuntaba a éste.
 - Que tenga un único hijo.
 - El elemento anterior se enlaza con el hijo del que se requiere borrar.
 - Que tenga dos hijos.
 - Se sustituye por el elemento más próximo inmediato superior o inmediato inferior.
 - Para localizar estos elementos debe el elemento eliminado sustituirse por el nodo que se encuentra más a la izquierda en el subárbol derecho o por el nodo que se encuentra más a la derecha en el subárbol izquierdo.

Ejemplo

Eliminar 7,6,5



Árbol AVL

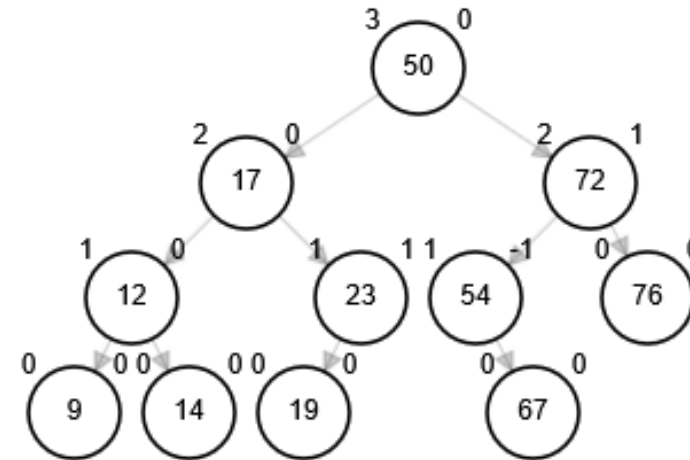
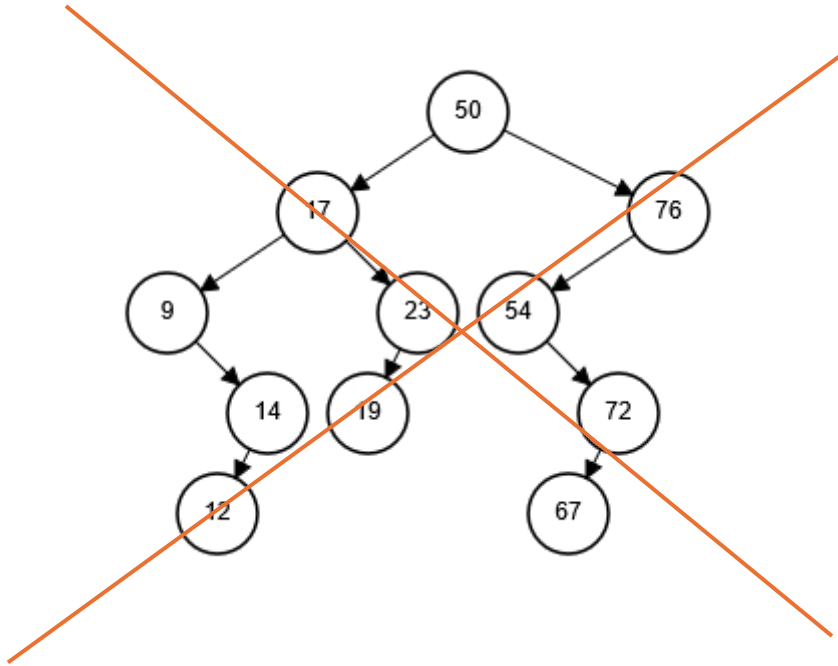
Básicamente un árbol AVL es un Árbol Binario de Búsqueda al que se le añade una condición de equilibrio.

“Para todo nodo la altura de sus subárboles izquierdo y derecho pueden diferir a lo sumo en 1”.

Gracias a esta forma de equilibrio (o balanceo), la complejidad de una búsqueda en uno de estos arboles se mantiene siempre en orden de complejidad $O(\log n)$

Condición de equilibrio

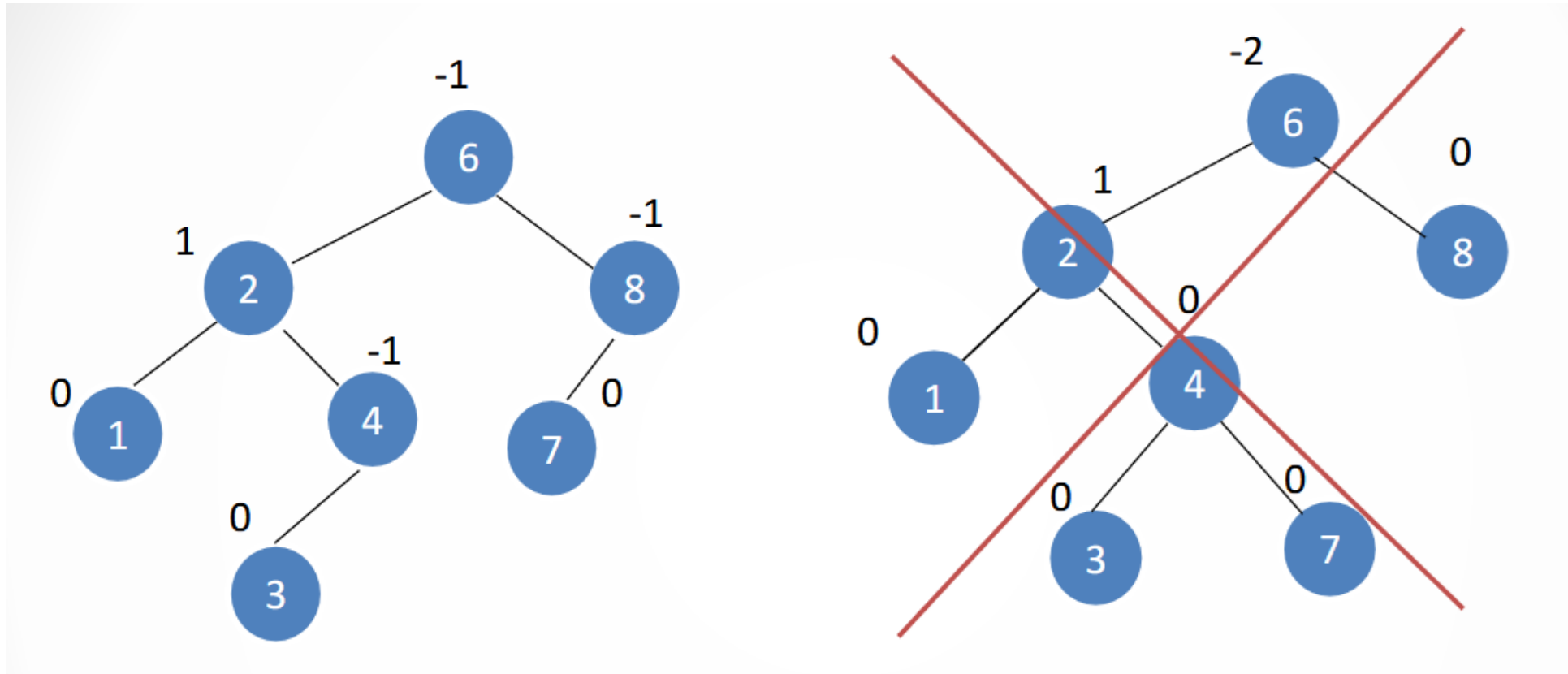
“Para todos los nodos, la altura de la rama izquierda no difiere en mas de una unidad de la altura de la rama derecha”



Características

- Un AVL es un ABB
- La diferencia entre las alturas de los subárboles derecho e izquierdo no debe excederse mas de 1.
- Cada nodo tiene asignado un peso de acuerdo con las alturas de sus subárboles. Un nodo tiene un peso de 1 si su subárbol derecho es mas alto, -1 si su subárbol izquierdo es mas alto y 0 si las alturas son las mismas.
- La inserción y eliminación en un árbol AVL es la misms que en un ABB.

Solo el árbol de la izquierda es AVL. El de la derecha viola la condición de equilibrio en el nodo 6, ya que su subárbol izquierdo tiene altura 3 y su subárbol derecho tiene altura 1.



Equilibrio

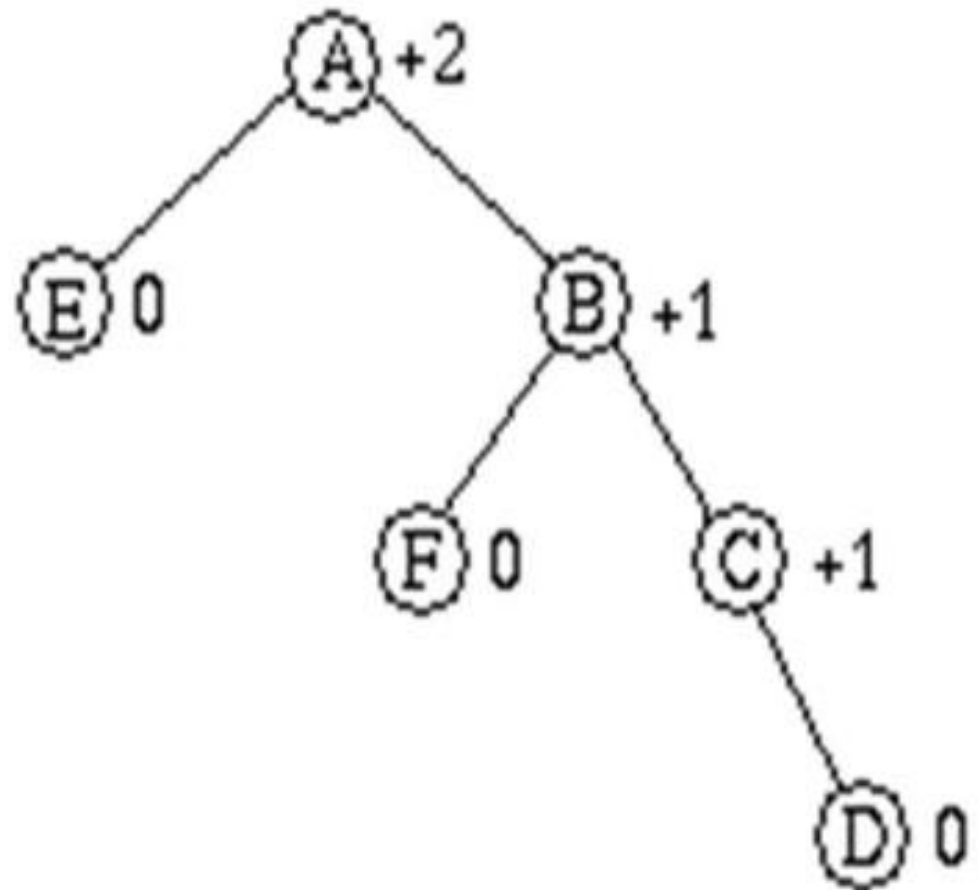
- **Equilibrio** = (altura derecha) - (altura izquierda)
- Describe relatividad entre subárbol derecho y izquierdo
 - + (Positivo) derecha mas alto (profundo)
 - - (Negativo) izquierda mas alto (profundo)

“Un árbol binario es un AVL si y solo si cada uno de sus nodos tiene un equilibrio de -1,0,+1”

- Si alguno de los pesos de los nodos se modifica en un valor no válido (2 o -2) debe seguirse un esquema de rotación.

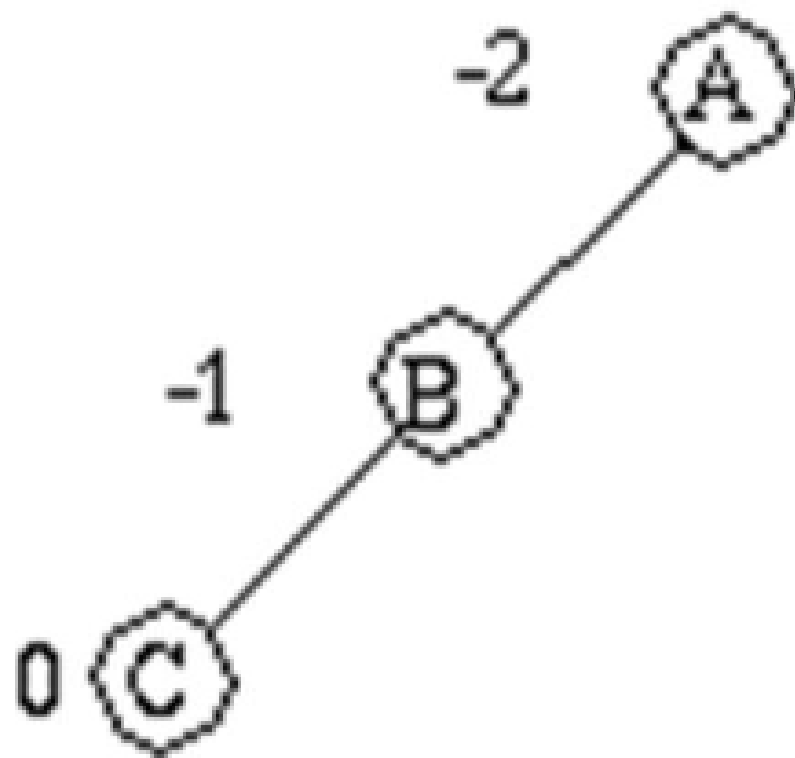
Desequilibrio hacia la izquierda
(Equilibrio $> +1$)

Desequilibrios



Desequilibrio hacia la derecha
(Equilibrio < -1)

Desequilibrio S



Operaciones sobre un AVL

Insertar Nodo

Balancear

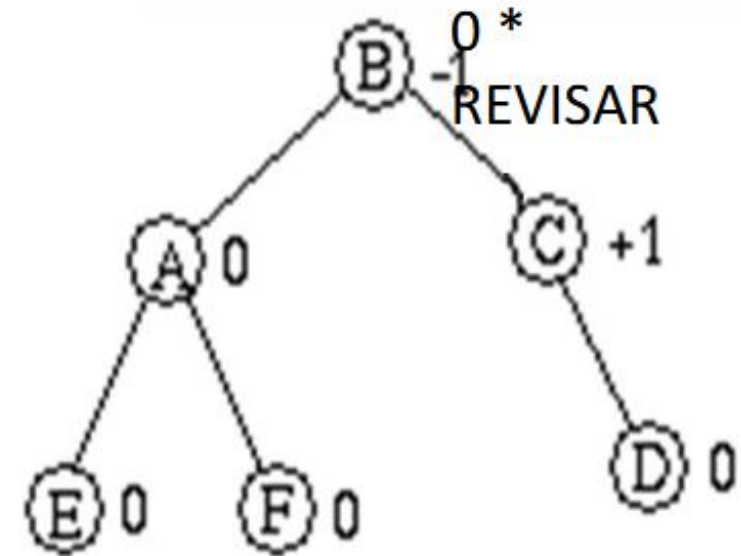
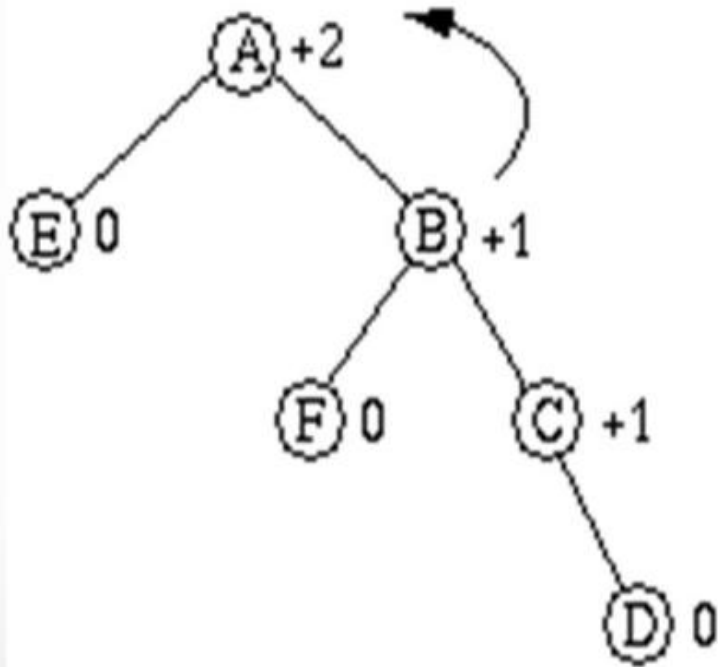
- Caso 1: Rotación simple izquierda RSI
- Caso 2: Rotación simple derecha RSD
- Caso 3: Rotación doble izquierda RDI
- Caso 4: Rotación doble derecha RDD

Eliminar nodo

Calcular altura

Insertar un nodo

1. Se usa la misma técnica que para insertar un nodo en un ABB ordenado
2. Trazamos una ruta desde el nodo raíz hasta un nodo hoja (donde hacemos la inserción).
3. Insertamos el nodo nuevo.
4. Volvemos a trazar la ruta de regreso al nodo raíz, ajustando el equilibrio a lo largo de ella.
5. Si el equilibrio de un nodo llega a ser $+ - 2$, volvemos a ajustar los subárboles de los nodos para que su equilibrio se mantenga acorde con los lineamientos AVL (que son ± 1)



Balancear

- Caso 1: Rotación simple izquierda RSI
 - Si esta desequilibrado a la izquierda ($E > +1$) y su hijo derecho tiene el mismo signo (+) hacemos rotación sencilla izquierda.